

"""

R — Trend-Following Trading Bot

=====

R اسم البوت:

والحجم. RSI مع تأكيد من مؤشر (EMA Crossover) استراتيجية: تقاطع المتوسطات المتحركة (مش في منطقة تشبع شرائي + حجم تداول مؤكد. RSI - شراء: لما المتوسط السريع يقطع المتوسط البطيء لأعلى + - بيع/إغلاق: لما المتوسط السريع يقطع المتوسط البطيء لأسفل، أو يتحقق وقف الخسارة/جني الأرباح.

إدارة المخاطر:

(متوسط المدى الحقيقي) — يتكيف مع تقلب السوق. ATR بناءً على (Stop Loss - وقف خسارة)
(Risk:Reward بنسبة من المخاطرة) (Take Profit - جني أرباح)
- حجم الصفقة محسوب تلقائيًا بحيث لا تتجاوز المخاطرة نسبة معينة من رأس المال (مثلاً 1%).
- حد أقصى لعدد الصفقات المفتوحة، وحد أقصى للخسارة اليومية (يوقف البوت تلقائيًا).

⚠ تنويه: هذا الكود لأغراض تعليمية/برمجية. ليس نصيحة مالية أو استثمارية.

قبل أي تفعيل حقيقي. PAPER التداول الحقيقي يحمل مخاطر خسارة كاملة لرأس المال. اختبر جيدًا في وضع

"""

```
import time
```

```
import logging
```

```
import math
```

```
from dataclasses import dataclass, field
```

```
from datetime import datetime, timezone
```

```
from typing import Optional
```

```
import numpy as np
```

```
import pandas as pd
```

```
try:
```

```
    import ccxt # (Binance, Kraken, ...) مكتبة موحدة للتعامل مع أغلب منصات التداول
```

```
except ImportError:
```

```
    ccxt = None
```

```
# =====
```

```
) #Config) الإعدادات
```

```
# =====
```

```
@dataclass
```

```
class BotConfig:
```

```
    exchange_id: str = "binance" # (ccxt) اسم المنصة
```

```
    symbol: str = "BTC/USDT" # زوج التداول
```

```
    timeframe: str = "15m" # الفريم الزمني للشموع
```

```
    fast_ema: int = 12 # المتوسط السريع
```

```
    slow_ema: int = 26 # المتوسط البطيء
```

```
    rsi_period: int = 14
```

```
    rsi_overbought: float = 75.0
```

```
    rsi_oversold: float = 25.0
```

```
    volume_ma_period: int = 20 # لتأكيد الحجم
```

```
    atr_period: int = 14
```

```
    atr_stop_multiplier: float = 2.0 # هذا الرقم * ATR المسافة بين وقف الخسارة والدخول =
```

```
    risk_reward_ratio: float = 2.0 # جني الأرباح = المخاطرة * هذا الرقم
```

```
    risk_per_trade_pct: float = 1.0 # من رأس المال يتم المخاطرة به في كل صفقة %
```

```
max_daily_loss_pct: float = 5.0    # يوقف البوت لو الخسارة اليومية تجاوزت هذه النسبة
max_open_positions: int = 1
poll_seconds: int = 30             # كل قد إيه يفحص البوت السوق
paper_trading: bool = True         # وضع تجريبي بدون تنفيذ حقيقي (الافتراضي والموصى به)
starting_balance: float = 10_000.0 # PAPER رأس مال افتراضي لوضع
api_key: Optional[str] = None
api_secret: Optional[str] = None
log_file: str = "trend_bot.log"
```

```
# =====
) #Logging) إدارة السجلات
# =====
```

```
def setup_logger(cfg: BotConfig) -> logging.Logger:
```

```
    logger = logging.getLogger("trend_bot")
```

```
    logger.setLevel(logging.INFO)
```

```
    logger.handlers.clear()
```

```
    fmt = logging.Formatter("%(asctime)s | %(levelname)-7s | %(message)s", "%Y-%m-%d
%H:%M:%S")
```

```
    console = logging.StreamHandler()
```

```
    console.setFormatter(fmt)
```

```
    logger.addHandler(console)
```

```
file_handler = logging.FileHandler(cfg.log_file, encoding="utf-8")
```

```
file_handler.setFormatter(fmt)
```

```
logger.addHandler(file_handler)
```

```
return logger
```

```
# =====
```

```
) #Technical Indicators المؤشرات الفنية
```

```
# =====
```

```
def ema(series: pd.Series, period: int) -> pd.Series:
```

```
    return series.ewm(span=period, adjust=False).mean()
```

```
def rsi(series: pd.Series, period: int) -> pd.Series:
```

```
    delta = series.diff()
```

```
    gain = delta.clip(lower=0)
```

```
    loss = -delta.clip(upper=0)
```

```
    avg_gain = gain.ewm(alpha=1 / period, adjust=False).mean()
```

```
    avg_loss = loss.ewm(alpha=1 / period, adjust=False).mean()
```

```
    rs = avg_gain / avg_loss.replace(0, np.nan)
```

```
    result = 100 - (100 / (1 + rs))
```

```
    return result.fillna(50)
```

```

def atr(df: pd.DataFrame, period: int) -> pd.Series:
    high, low, close = df["high"], df["low"], df["close"]
    prev_close = close.shift(1)
    tr = pd.concat([
        high - low,
        (high - prev_close).abs(),
        (low - prev_close).abs(),
    ], axis=1).max(axis=1)
    return tr.ewm(alpha=1 / period, adjust=False).mean()

```

```

def add_indicators(df: pd.DataFrame, cfg: BotConfig) -> pd.DataFrame:
    df = df.copy()
    df["ema_fast"] = ema(df["close"], cfg.fast_ema)
    df["ema_slow"] = ema(df["close"], cfg.slow_ema)
    df["rsi"] = rsi(df["close"], cfg.rsi_period)
    df["atr"] = atr(df, cfg.atr_period)
    df["volume_ma"] = df["volume"].rolling(cfg.volume_ma_period).mean()
    return df

```

```

# =====

```

```

) حالة الصفقة #Position State)

```

```

# =====

```

```

@dataclass

```

```

class Position:

    side: str          # "long"

    entry_price: float

    size: float        # كمية الأصل

    stop_loss: float

    take_profit: float

    opened_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))

```

```

# =====
# البوت الأساسي
# =====

```

```

class R:

    """Trend-Following تداول اتجاهي ( R — بوت """

```

```

    def __init__(self, cfg: BotConfig):

        self.cfg = cfg

        self.logger = setup_logger(cfg)

        self.exchange = self._init_exchange()

        self.balance = cfg.starting_balance

        self.position: Optional[Position] = None

        self.daily_pnl = 0.0

        self.daily_start_balance = cfg.starting_balance

        self.day_marker = datetime.now(timezone.utc).date()

```

```
self.halted = False
```

```
----- الاتصال بالمنصة ----- #

def _init_exchange(self):

    if self.cfg.paper_trading:

        self.logger.info(" (Paper Trading) وضع التشغيل: تداول تجريبي ")
        لن يتم تنفيذ أوامر حقيقية.

    if ccxt is None:

        self.logger.warning("ccxt مكتبة ")
        غير مثبتة؛ سيتم توليد بيانات تجريبية بدلاً من جلب بيانات حقيقية.

        return None

    # فقط لجلب بيانات السوق الحقيقية، بدون تنفيذ أوامر ccxt
    exchange_class = getattr(ccxt, self.cfg.exchange_id)

    return exchange_class({"enableRateLimit": True})

    if ccxt is None:

        raise RuntimeError("pip install ccxt مطلوبة للتداول الحقيقي. ثبتها عبر: ccxt مكتبة ")

    if not self.cfg.api_key or not self.cfg.api_secret:

        raise RuntimeError(
            "API. "
            "التداول الحقيقي مفعل لكن لا يوجد مفاتيح
            قبل التشغيل الحقيقي. " BotConfig في api_secret و api_key
            "من فضلك عَيِّن
            )

        self.logger.warning("⚠ وضع التشغيل: تداول حقيقي — سيتم تنفيذ أوامر فعلية بأموال حقيقية!")

        exchange_class = getattr(ccxt, self.cfg.exchange_id)

        return exchange_class({

            "apiKey": self.cfg.api_key,

            "secret": self.cfg.api_secret,
```

```
"enableRateLimit": True,  
})
```

----- جلب بيانات السوق ----- #

```
def fetch_market_data(self, limit: int = 200) -> pd.DataFrame:  
    if self.exchange is not None:  
        ohlcv = self.exchange.fetch_ohlcv(self.cfg.symbol, timeframe=self.cfg.timeframe,  
limit=limit)  
        df = pd.DataFrame(ohlcv, columns=["timestamp", "open", "high", "low", "close",  
"volume"])  
        df["timestamp"] = pd.to_datetime(df["timestamp"], unit="ms")  
        return df  
  
# fallback: غير متاح — للتجربة المحلية بدون إنترنت) ccxt بيانات وهمية عشوائية (فقط لو  
return self._generate_fake_data(limit)
```

```
def _generate_fake_data(self, limit: int) -> pd.DataFrame:  
    rng = np.random.default_rng()  
    prices = 100 + np.cumsum(rng.normal(0, 1, limit))  
    prices = np.clip(prices, 1, None)  
    df = pd.DataFrame({  
        "timestamp": pd.date_range(end=datetime.now(), periods=limit, freq="15min"),  
        "open": prices,  
        "high": prices + rng.uniform(0, 1, limit),  
        "low": prices - rng.uniform(0, 1, limit),  
        "close": prices + rng.normal(0, 0.3, limit),  
        "volume": rng.uniform(50, 200, limit),
```

```
}}
```

```
return df
```

```
----- منطق القرار ----- #
```

```
def generate_signal(self, df: pd.DataFrame) -> str:
```

```
'بناءً على آخر شمعتين مكتملتين. hold' أو 'sell' أو 'buy' "" "" "" ""يرجع'
```

```
last, prev = df.iloc[-1], df.iloc[-2]
```

```
crossed_up = prev["ema_fast"] <= prev["ema_slow"] and last["ema_fast"] >  
last["ema_slow"]
```

```
crossed_down = prev["ema_fast"] >= prev["ema_slow"] and last["ema_fast"] <  
last["ema_slow"]
```

```
volume_confirmed = last["volume"] > last["volume_ma"] if not  
math.isnan(last["volume_ma"]) else False
```

```
rsi_ok_for_buy = last["rsi"] < self.cfg.rsi_overbought
```

```
rsi_ok_for_sell = last["rsi"] > self.cfg.rsi_oversold
```

```
if crossed_up and volume_confirmed and rsi_ok_for_buy:
```

```
    return "buy"
```

```
if crossed_down and rsi_ok_for_sell:
```

```
    return "sell"
```

```
return "hold"
```

```
----- إدارة المخاطر ----- #
```

```
def calculate_position_size(self, entry_price: float, stop_loss: float) -> float:
```

```
من رأس المال. "" "" risk_per_trade_pct "" ""يحسب حجم الصفقة بحيث لا تتجاوز الخسارة المحتملة نسبة
```

```

risk_amount = self.balance * (self.cfg.risk_per_trade_pct / 100)

stop_distance = abs(entry_price - stop_loss)

if stop_distance <= 0:

    return 0.0

size = risk_amount / stop_distance

return size

```

```

def reset_daily_tracking_if_needed(self):

    today = datetime.now(timezone.utc).date()

    if today != self.day_marker:

        self.day_marker = today

        self.daily_start_balance = self.balance

        self.daily_pnl = 0.0

        self.halted = False

        self.logger.info("بداية يوم تداول جديد — إعادة تعيين تتبع الخسارة اليومية.")

```

```

def check_daily_loss_limit(self) -> bool:

    loss_pct = ((self.daily_start_balance - self.balance) / self.daily_start_balance) * 100

    if loss_pct >= self.cfg.max_daily_loss_pct:

        if not self.halted:

            self.logger.warning(

                f"⛔ {loss_pct:.2f}%). "

                "إيقاف البوت عن فتح صفقات جديدة حتى اليوم التالي."

            )

            self.halted = True

    return self.halted

```

----- تنفيذ الأوامر ----- #

```
def open_long(self, price: float, atr_value: float):
```

```
    stop_loss = price - (atr_value * self.cfg.atr_stop_multiplier)
```

```
    risk = price - stop_loss
```

```
    take_profit = price + (risk * self.cfg.risk_reward_ratio)
```

```
    size = self.calculate_position_size(price, stop_loss)
```

```
    if size <= 0:
```

```
        self.logger.warning("حجم الصفقة المحسوب = 0، تم تجاهل الإشارة.")
```

```
        return
```

```
    if self.cfg.paper_trading:
```

```
        self.logger.info(
```

```
            f"📈 {self.cfg.symbol} | السعر: {price:.2f} | الكمية: {size:.6f} | شراء [تجريبي]"
```

```
            f" | أرباح: {take_profit:.2f} | وقف خسارة: {stop_loss:.2f} |]"
```

```
        )
```

```
    else:
```

```
        order = self.exchange.create_market_buy_order(self.cfg.symbol, size)
```

```
        self.logger.info(f"✅ {self.cfg.symbol} | order تم تنفيذ أمر شراء حقيقي")
```

```
    self.position = Position(side="long", entry_price=price, size=size,
```

```
                             stop_loss=stop_loss, take_profit=take_profit)
```

```
def close_position(self, price: float, reason: str):
```

```
    if self.position is None:
```

```
return
```

```
pnl = (price - self.position.entry_price) * self.position.size
```

```
self.balance += pnl
```

```
self.daily_pnl += pnl
```

```
if self.cfg.paper_trading:
```

```
    self.logger.info(
```

```
        f"📉 {reason} [تجريبي] إغلاق صفقة (price:.2f) "
```

```
        f"| {self.balance:.2f} الرصيد الحالي: {pnl:+.2f} الربح/الخسارة: {pnl:.2f} "
```

```
    )
```

```
else:
```

```
    order = self.exchange.create_market_sell_order(self.cfg.symbol, self.position.size)
```

```
    self.logger.info(f"✅ {order} تم تنفيذ أمر بيع حقيقي")
```

```
self.position = None
```

```
----- # الحلقة الرئيسية -----
```

```
def check_open_position(self, current_price: float):
```

```
    """يتحقق من وقف الخسارة/جني الأرباح للصفقة المفتوحة."""
```

```
    if self.position is None:
```

```
        return
```

```
    if current_price <= self.position.stop_loss:
```

```
        self.close_position(current_price, "وقف خسارة")
```

```
    elif current_price >= self.position.take_profit:
```

```
self.close_position(current_price, "جني أرباح")
```

```
def run_once(self):
```

```
    self.reset_daily_tracking_if_needed()
```

```
    df = self.fetch_market_data()
```

```
    df = add_indicators(df, self.cfg)
```

```
    df = df.dropna().reset_index(drop=True)
```

```
    if len(df) < 2:
```

```
        self.logger.warning("بيانات غير كافية لاتخاذ قرار.")
```

```
    return
```

```
    current_price = df.iloc[-1]["close"]
```

```
    current_atr = df.iloc[-1]["atr"]
```

```
# 1 تحقق من الصفقة المفتوحة أولاً (وقف خسارة/جني أرباح)
```

```
    self.check_open_position(current_price)
```

```
# 2 لو البوت متوقف بسبب حد الخسارة اليومية، لا يفتح صفقات جديدة
```

```
    if self.check_daily_loss_limit():
```

```
        return
```

```
# 3 لو لا توجد صفقة مفتوحة ولم نتجاوز الحد الأقصى، افحص الإشارة
```

```
    if self.position is None:
```

```
        signal = self.generate_signal(df)
```

```

if signal == "buy":
    self.open_long(current_price, current_atr)
else:
    self.logger.info(f"{current_price:.2f} | RSI: {df.iloc[-1]['rsi']:.1f}")
else:
    self.logger.info(
        f"{current_price:.2f} | السعر الحالي: {current_price:.2f} "
        f"| self.position.entry_price:.2f | self.position.stop_loss:.2f | "
        f"الدخول: {current_price:.2f} | "
    )

```

```

def run_forever(self):

```

```

    self.logger.info(
        f"{self.cfg.timeframe} | الفريم: {self.cfg.symbol} | الزوج: {self.cfg.symbol} | R بدء تشغيل البوت  "
        f"حقيقي  '{self.cfg.paper_trading}' | وضع: {self.cfg.paper_trading} | تجريبي: '{self.cfg.paper_trading}' "
    )

```

```

try:

```

```

    while True:

```

```

        try:

```

```

            self.run_once()

```

```

        except Exception as e:

```

```

            self.logger.exception(f"{e} أثناء التشغيل: {e}")

```

```

            time.sleep(self.cfg.poll_seconds)

```

```

except KeyboardInterrupt:

```

```

    self.logger.info("تم إيقاف البوت يدوياً.")

```

```
# =====
```

```
# نقطة التشغيل
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    config = BotConfig(
```

```
        exchange_id="binance",
```

```
        symbol="BTC/USDT",
```

```
        timeframe="15m",
```

```
        paper_trading=True, # لا تحوله لـ True اتركه # False للتجربة. إلا بعد اختبار كافٍ ووعي تام بالمخاطر.
```

```
        risk_per_trade_pct=1.0,
```

```
        max_daily_loss_pct=5.0,
```

```
    )
```

```
    bot = R(config)
```

```
    bot.run_forever()
```